SESSION 12

Assessment Submission

Advanced Theming, Animation Design & Polished UI Module

Student: Sudarshan G

Course: Android App Development with Kotlin & Jetpack Compose

Session: 12 — Material 3 Theming, Animation APIs & Integrated UI Module

Tasks: Task 1 · Task 2 · Task 3 · Task 4

TASK 1 — Advanced Theming Strategy: Material 3 + Brand Identity

1. Research Theming Concepts

1.1 Core Definitions

colorScheme in Material 3

The colorScheme is a structured set of named colour roles (primary, secondary, tertiary, background, surface, error, and their on-* counterparts) that defines the complete colour identity of a Material 3 app. Components reference these named roles rather than raw hex values, so changing the colorScheme propagates the brand update to every composable automatically without touching individual files.

typography in Material 3

The Typography object defines a set of named text styles (displayLarge, headlineMedium, titleSmall, bodyMedium, labelSmall, etc.) covering every text use case in the app. Each style specifies font family, font weight, font size, line height, and letter spacing as a cohesive system. Composables reference typography tokens (e.g., MaterialTheme.typography.headlineMedium) so all text scales and font choices update consistently from one place.

shapes in Material 3

The Shapes object defines three corner radius tokens — small, medium, and large — that control the roundness of interactive components like buttons, cards, dialogs, and chips. Using shape tokens ensures visual consistency: changing MaterialTheme.shapes.medium from 12dp to 16dp globally updates every Card and OutlinedTextField that references that token, without touching individual composables.

Dynamic Colour (Android 12+ / Material You)

Dynamic colour is a Material You feature available on Android 12 and above that generates a personalised ColorScheme by extracting the five key colours from the user's current wallpaper. The system provides these colours via dynamicDarkColorScheme() and dynamicLightColorScheme(), allowing the app's entire colour system to harmonise with the user's device aesthetic. On Android 11 and below, the app falls back to a static brand-defined colorScheme.

Light / Dark Theme Support

Light and dark theme support means providing two distinct ColorScheme instances — one with light backgrounds and dark text, one with dark backgrounds and light text — and switching between them based on the system's current UI mode. In Compose, isSystemInDarkTheme() reads the system setting and the app passes the appropriate colorScheme to MaterialTheme. This allows the app to respect the user's battery-saving and visual-comfort preferences automatically.

1.2 Code Examples for Each Concept

Example 1 — Declaring a static ColorScheme:

```
// Color.kt
val WanderLightColorScheme = lightColorScheme(
    primary          = Color(0xFF1565C0),
    onPrimary        = Color(0xFFFFFFFF),
    primaryContainer = Color(0xFFD0E4FF),
    secondary        = Color(0xFF00897B),
    onSecondary      = Color(0xFFFFFFFF),
    tertiary         = Color(0xFFF57F17),
    background       = Color(0xFFF8FAFB),
    surface          = Color(0xFFFFFFFF),
    error            = Color(0xFFB3261E),
    onBackground     = Color(0xFF1A1A2E),
    onSurface        = Color(0xFF1A1A2E)
)

val WanderDarkColorScheme = darkColorScheme(
    primary          = Color(0xFF90CAF9),
    onPrimary        = Color(0xFF003064),
    primaryContainer = Color(0xFF1565C0),
    secondary        = Color(0xFF80CBC4),
    tertiary         = Color(0xFFFFCC02),
    background       = Color(0xFF0D0D1A),
    surface          = Color(0xFF1A1A2E),
    error            = Color(0xFFF2B8B5),
    onBackground     = Color(0xFFE8EAF6),
    onSurface        = Color(0xFFE8EAF6)
)
```

Example 2 — Typography token declaration:

```
// Typography.kt
val WanderTypography = Typography(
    displayLarge  = TextStyle(fontFamily = MontserratFamily, fontWeight =
FontWeight.Bold,
                              fontSize = 57.sp, lineHeight = 64.sp),
    headlineMedium = TextStyle(fontFamily = MontserratFamily, fontWeight =
FontWeight.SemiBold,
                              fontSize = 28.sp, lineHeight = 36.sp),
    titleLarge    = TextStyle(fontFamily = MontserratFamily, fontWeight =
FontWeight.Medium,
                              fontSize = 22.sp, lineHeight = 28.sp),
    bodyMedium    = TextStyle(fontFamily = RobotoFamily, fontWeight = FontWeight.Normal,
                              fontSize = 14.sp, lineHeight = 20.sp),
    labelSmall    = TextStyle(fontFamily = RobotoFamily, fontWeight = FontWeight.Medium,
                              fontSize = 11.sp, lineHeight = 16.sp, letterSpacing =
0.5.sp)
)
```

Example 3 — Dynamic colour with light/dark fallback:

```
// Theme.kt
@Composable
fun WanderAppTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context)
            else dynamicLightColorScheme(context)
        }
        darkTheme -> WanderDarkColorScheme
        else      -> WanderLightColorScheme
    }
    MaterialTheme(
        colorScheme = colorScheme,
        typography  = WanderTypography,
        shapes      = WanderShapes,
        content     = content
    )
}
```

2. Brand + Theme Design

2.1 App Brand — WanderApp

App Name: WanderApp

WanderApp is a travel discovery and trip-planning app. The brand is built around the feeling of horizon, open skies, and calm confidence — a traveller who plans thoughtfully and explores with purpose. The brand identity uses deep ocean blue as the primary colour (evoking trust, travel, and depth), teal-green as the secondary (adventure, nature), and amber as the tertiary accent (warmth, sunset highlights). The wordmark uses Montserrat Bold for display and Roboto for body text.

2.2 Complete Colour Palette — Light & Dark

Token	Light Mode Hex	Light Role	Dark Mode Hex	Dark Role
primary	#1565C0	Buttons, AppBar, FAB	#90CAF9	Primary text/icons on dark
onPrimary	#FFFFFF	Text on primary	#003064	Text on dark primary
primaryContainer	#D0E4FF	Highlighted chips/cards	#1565C0	Container on dark
secondary	#00897B	Toggles, active states	#80CBC4	Secondary accents on dark

Token	Light Mode Hex	Light Role	Dark Mode Hex	Dark Role
onSecondary	#FFFFFF	Text on secondary	#003731	Text on dark secondary
tertiary	#F57F17	Tags, badges, accents	#FFCC02	Warm accent on dark
background	#F8FAFB	Screen background	#0D0D1A	Dark screen bg
surface	#FFFFFF	Cards, sheets, dialogs	#1A1A2E	Dark surface
onSurface	#1A1A2E	Text/icons on surface	#E8EAF6	Text on dark surface
surfaceVariant	#DDE3EA	Expanded card bg	#3A3A4A	Expanded card on dark
error	#B3261E	Error states	#F2B8B5	Error on dark
outline	#73777F	Input borders, dividers	#8D9199	Borders on dark

2.3 Typography Tokens

Token Name	Material 3 Role	Font Family	Weight	Size	Use Case
displayLarge	displayLarge	Montserrat	Bold 700	57sp	Hero onboarding headlines
headlineMedium	headlineMedium	Montserrat	SemiBold 600	28sp	Screen titles, section headers
titleLarge	titleLarge	Montserrat	Medium 500	22sp	Card titles, AppBar text
titleMedium	titleMedium	Montserrat	Medium 500	16sp	List item titles
bodyMedium	bodyMedium	Roboto	Regular 400	14sp	Descriptions, paragraphs
bodySmall	bodySmall	Roboto	Regular 400	12sp	Captions, secondary info
labelMedium	labelMedium	Roboto	Medium 500	12sp	Chip labels, tab labels
labelSmall	labelSmall	Roboto	Medium 500	11sp	Badges, fine print

2.4 Shape Tokens

Token	Corner Radius	Components That Use It	Visual Effect
shapes.extraSmall	4dp	Tooltips, small badges	Slightly softened corners
shapes.small	8dp	AssistChip, FilterChip, TextFields	Rounded chips and inputs
shapes.medium	12dp	Cards, AlertDialog, SearchBar	Soft rounded card corners
shapes.large	16dp	BottomSheet, NavigationDrawer	Prominent rounding
shapes.extraLarge	28dp	FAB, large action buttons	Pill-like capsule buttons
shapes.full	50% (circle)	IconButton, Avatar images	Perfect circular elements

```
// Shapes.kt
val WanderShapes = Shapes(
    extraSmall  = RoundedCornerShape(4.dp),
    small       = RoundedCornerShape(8.dp),
    medium      = RoundedCornerShape(12.dp),
    large       = RoundedCornerShape(16.dp),
    extraLarge  = RoundedCornerShape(28.dp)
)
```

3. Theme Architecture Document

3.1 File Structure

File	Location	Responsibility
Color.kt	ui/theme/Color.kt	Defines all Color values (hex); WanderLightColorScheme; WanderDarkColorScheme
Typography.kt	ui/theme/Typography.kt	WanderTypography object; font family declarations; TextStyle values
Shapes.kt	ui/theme/Shapes.kt	WanderShapes object; all corner radius tokens
Theme.kt	ui/theme/Theme.kt	WanderAppTheme composable; dynamic colour logic; isSystemInDarkTheme()
Type.kt (fonts)	ui/theme/Type.kt	FontFamily declarations (MontserratFamily, RobotoFamily); font loading

3.2 MaterialTheme Wrapper

```
// MainActivity.kt — app entry point
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            WanderAppTheme {  // <-- all theme tokens injected here
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    WanderNavHost()  // Navigation + all screens inside
                }
            }
        }
    }
}

// WanderAppTheme wraps everything — no composable below ever uses
// raw Color() or hardcoded TextStyle; they all call MaterialTheme.colorScheme.*
```

3.3 How Components Reference Theme Tokens

Instead of writing backgroundColor = Color(0xFF1565C0) inside a composable, every component reads from MaterialTheme: background = MaterialTheme.colorScheme.primary. This means the component works correctly in light mode, dark mode, and dynamic colour mode without any changes. The MaterialTheme object is a CompositionLocal — it is implicitly available to every composable in the tree below the WanderAppTheme wrapper, accessed via MaterialTheme.colorScheme, MaterialTheme.typography, and MaterialTheme.shapes.

4. Theming Across UI Components

Component	colorScheme Token	Typography Token	Shape Token	Elevation / Other
TopAppBar	containerColor = colorScheme.primary; titleContentColor = colorScheme.onPrimary	typography.titleLarge for title text	N/A (no shape)	shadowElevation = 4dp; NavigationIcon tint = onPrimary
DestinationCard	containerColor = colorScheme.surface; expanded bg = colorScheme.surfaceVariant	titleLarge for city name; bodySmall for description	shapes.medium (12dp corners)	elevation = 4dp collapsed, 8dp expanded
FilledButton (primary action)	containerColor = colorScheme.primary; contentColor = colorScheme.onPrimary	labelLarge for button text	shapes.extraLarge (28dp — pill)	No shadow; ripple uses colorScheme.primary
OutlinedTextField (search/input)	focusedBorderColor = colorScheme.primary; errorBorderColor = colorScheme.error	bodyMedium for input text; labelMedium for floating label	shapes.small (8dp)	No elevation; cursor = colorScheme.primary

Component	colorScheme Token	Typography Token	Shape Token	Elevation / Other
FilterChip (active)	selectedContainerColor = colorScheme.secondaryContainer; labelColor = colorScheme.onSecondaryContainer	labelMedium for chip text	shapes.small (8dp)	Checkmark icon uses colorScheme.secondary
NavigationBar	containerColor = colorScheme.surfaceContainer; indicatorColor = colorScheme.secondaryContainer	labelSmall for nav labels	N/A (full-width bar)	elevation = 0dp (no shadow per M3)

```
// Example: DestinationCard with theme tokens
@Composable
fun DestinationCard(destination: TravelDestination, isExpanded: Boolean) {
    val cardColor by animateColorAsState(
        targetValue = if (isExpanded)
            MaterialTheme.colorScheme.surfaceVariant
        else
            MaterialTheme.colorScheme.surface,
        label = "card_bg_color"
    )
    Card(
        shape     = MaterialTheme.shapes.medium,       // 12dp from WanderShapes
        colors    = CardDefaults.cardColors(containerColor = cardColor),
        elevation = CardDefaults.cardElevation(
            defaultElevation = if (isExpanded) 8.dp else 4.dp
        )
    ) {
        Column {
            Text(
                text  = destination.cityName,
                style = MaterialTheme.typography.titleLarge,  // from WanderTypography
                color = MaterialTheme.colorScheme.onSurface
            )
            Text(
                text  = destination.description,
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
        }
    }
}
```

5. Reflection

A well-structured theming system is essential for scalability because it decouples visual design decisions from component implementation. When the brand updates its primary colour — perhaps for a seasonal campaign or a rebrand — a single change in Color.kt propagates to every button, app bar, chip, and interactive element across the entire app simultaneously, without touching any composable

files. By contrast, an app with hard-coded colour values scattered across dozens of composables requires a manual search-and-replace across every file, which is slow, error-prone, and inevitably produces inconsistencies where some components are updated and others are missed. The result of inconsistent hard-coded styles is a UI that looks different on different screens — buttons with slightly different blues, cards with mismatched corner radii — which erodes user trust and makes the app feel unpolished. Dynamic colour and light/dark theming further improve user experience by personalising the app to each individual device. On Android 12+ with a sunset wallpaper, WanderApp's palette adapts to warm amber tones that feel harmonious with the device environment, creating a cohesive visual experience that feels native rather than generic. Dark mode, beyond aesthetics, actively reduces battery consumption on OLED displays and reduces eye strain in low-light environments, making the app more comfortable for extended use. Together, these theming features transform an app from a collection of screens into a coherent, brand-consistent product that respects and responds to user preferences.

TASK 2 — Animation Design & Planning in Compose

1. Research Animation APIs

1.1 Core Definitions

animateFloatAsState

animateFloatAsState is a Compose animation API that smoothly interpolates a Float value from its current value to a new target value whenever the target changes. It returns a State<Float> that can be read in the composable and applied to modifiers like Modifier.graphicsLayer(scaleX = scale), Modifier.alpha(alpha), or any other float-based property. The animation spec (spring, tween, or keyframes) is specified as a parameter and controls the timing and easing of the interpolation.

animateColorAsState

animateColorAsState smoothly transitions a Color value from its current state to a new target Color whenever the target changes, correctly interpolating through the colour space rather than jumping abruptly. It is commonly used to animate background colours, icon tints, border colours, and text colours in response to state changes like selected/unselected, expanded/collapsed, or active/inactive. Like all animate*AsState APIs, it is driven by the target state and handles interruptions gracefully — if the target changes mid-animation, the animation reverses from the current interpolated position.

AnimatedVisibility

AnimatedVisibility is a high-level Compose composable that animates the appearance and disappearance of its content using enter and exit transition specifications. When the visible parameter changes from false to true, the content slides, fades, or scales in according to the enter parameter; when it changes to false, the exit animation plays and the content is removed from the layout after the animation completes. Unlike if (condition) { Content() } which jumps instantly, AnimatedVisibility gives the UI time to communicate the state change to the user through motion.

animateContentSize

animateContentSize is a Modifier that animates layout size changes of a composable when its content changes dimensions. Instead of jumping abruptly to the new size when content is added or removed (e.g., when a card expands to show its full description), animateContentSize interpolates smoothly between the old and new sizes using the provided animationSpec. It is applied directly as a modifier: Modifier.animateContentSize(animationSpec = spring(...)), and requires no additional state tracking — it observes the measured size change automatically.

Transition (updateTransition)

updateTransition creates a Transition object that manages multiple animated values simultaneously, all driven by the same state variable. Instead of managing several separate animate*AsState calls (which can de-sync), a Transition ensures all animated properties progress together in lockstep — for example, a card's background colour, elevation, and content alpha all animate together when the card's expanded state changes. The transition.animateFloat(), transition.animateColor(), and transition.animateDp() extension functions define child animations that are all governed by the parent Transition state.

1.2 Code Example — animateColorAsState with updateTransition

```
// Using updateTransition to coordinate multiple animations
enum class CardState { COLLAPSED, EXPANDED }

@Composable
fun AnimatedDestinationCard(destination: TravelDestination, isExpanded: Boolean) {
    val cardState = if (isExpanded) CardState.EXPANDED else CardState.COLLAPSED
```

```
    val transition = updateTransition(targetState = cardState, label = "card_transition")

    // All these animate in lockstep with the same state change
    val backgroundColor by transition.animateColor(
        transitionSpec = { spring(stiffness = Spring.StiffnessMedium) },
        label = "card_bg"
    ) { state ->
        when (state) {
            CardState.COLLAPSED -> MaterialTheme.colorScheme.surface
            CardState.EXPANDED  -> MaterialTheme.colorScheme.surfaceVariant
        }
    }

    val elevation by transition.animateDp(
        transitionSpec = { tween(durationMillis = 200) },
        label = "card_elevation"
    ) { state -> if (state == CardState.EXPANDED) 8.dp else 4.dp }

    val contentAlpha by transition.animateFloat(
        transitionSpec = { tween(durationMillis = 300) },
        label = "content_alpha"
    ) { state -> if (state == CardState.EXPANDED) 1f else 0f }

    Card(
        colors    = CardDefaults.cardColors(containerColor = backgroundColor),
        elevation = CardDefaults.cardElevation(defaultElevation = elevation),
        modifier  = Modifier.animateContentSize(
            animationSpec = spring(dampingRatio = Spring.DampingRatioMediumBouncy)
        )
    ) {
        Column {
            // Always-visible header content
            CardHeader(destination)
            // Animated description — fades in with content size change
            if (isExpanded) {
                Text(
                    text     = destination.description,
                    modifier = Modifier.alpha(contentAlpha)
                )
            }
        }
    }
}
```

2. App Feature Selected for Animation

Feature: Travel Destination Card Gallery — Expand/Collapse with Favourite Toggle

The primary screen of WanderApp displays a scrollable gallery of destination cards. Each card starts in a compact collapsed state showing the hero image, city name, and rating. Tapping the card body expands it to reveal the full description, best season, and a 'View Details' button — the expansion uses animateContentSize for a smooth height transition with a background colour shift from surface to surfaceVariant. Tapping the heart icon triggers a satisfying scale bounce animation (keyframes) and a colour transition from white to red. When a new card enters the list (e.g., after filtering), it slides and

fades in using AnimatedVisibility. These three animations work together to create a UI that communicates state changes clearly — the user always knows what tapping will do and receives immediate, natural-feeling visual confirmation that their action was registered.

3. Animation Plan Document

#	Scenario	Trigger	Animation Type	UI Elements Affected	Duration / Spec
1	Card expand / collapse	Tap card body	animateContentSize + updateTransition	Card height, bg color (surface → surfaceVariant), content alpha (0→1)	spring(dampingRatio=MediumBouncy, stiffness=Low) ~300ms
2	Favourite icon toggle	Tap heart icon	animateFloatAsState (scale) + animateColorAsState (tint)	Icon scale 1.0→1.3→1.0; tint white→red	keyframes { 1.3f at 150ms; 1.0f at 300ms }
3	Card enter (after filter)	Filter chip tapped, list updates	AnimatedVisibility (enter)	Each card composable: fadeIn + slideInVertically	fadeIn(tween(300)) + slideInVertically(tween(250))
4	Card exit (after filter removes it)	Filter chip deselects item	AnimatedVisibility (exit)	Card composable: fadeOut + slideOutHorizontally	fadeOut(tween(200)) + slideOutHorizontally(tween(200))
5	Theme switch (dark/light toggle)	Settings switch toggled	animateColorAsState on all surface/bg composables	Background, card surface, text colour, icon tint	tween(durationMillis=400, easing=FastOutSlowInEasing)
6	Error field shake	Submit with invalid input	animateFloatAsState with keyframes on offsetX	OutlinedTextField translateX: 0→(-8)→8→(-4)→4→0	keyframes total 400ms
7	Rating stars reveal on expand	Card transition to EXPANDED state	AnimatedVisibility (enter)	Row of star icons below description	slideInHorizontally from start + fadeIn, tween 250ms, delay 150ms

4. Performance & Accessibility Notes

4.1 Performance Considerations

Performance Consideration 1: Avoid per-item animations in large LazyColumn lists

When a LazyColumn contains 50+ destination cards, running a complex Transition animation on every card simultaneously during scroll would cause significant frame drops. The solution is to limit animated properties to only what is visible and necessary: the currently expanded card animates its colour and content, while off-screen cards remain static. animateContentSize and updateTransition are applied per-card but only activate when the card's own state changes — not on every scroll event. Compose's recomposition scope means only the composable with the changed state rerenders; other cards remain unaffected as long as their state has not changed and they receive the same parameters.

Performance Consideration 2: Prefer graphicsLayer over layout-affecting modifiers for smooth animation

The favourite icon scale animation uses Modifier.graphicsLayer(scaleX = scale, scaleY = scale) rather than Modifier.size(animatedSize.dp). graphicsLayer performs the transformation on the GPU render layer without triggering a layout measurement pass, which is significantly cheaper at 60fps. Layout-affecting animations (those that change actual measured size) force a re-layout of the parent and all siblings on every frame, which can cause jank on complex layouts. Rule of thumb: use graphicsLayer for scale, rotation, alpha, and translation; use animateContentSize or updateTransition with Dp for genuine size changes where reflowing siblings is intentional.

Performance Consideration 3: Cache AnimationSpec instances

Creating a new Spring() or Tween() object on every recomposition allocates unnecessary objects that the garbage collector must clean up. Define animation specs as top-level val constants (val CardExpandSpec = spring(dampingRatio = MediumBouncy)) and reference them by name in animate*AsState calls. This ensures the same spec object is reused across all recompositions.

4.2 Accessibility & Motion Reduction Considerations

Accessibility Consideration 1: Respect system-level Reduce Motion setting

Android users can enable 'Remove Animations' or 'Reduce Animations' in Developer Options or Accessibility Settings. In Compose, the LocalAccessibilityManager or checking Settings.Global.ANIMATOR_DURATION_SCALE provides the current animation scale. A robust app reads this and conditionally replaces animated transitions with instant state changes. The recommended pattern is to define a composable-level flag (val animationsEnabled = LocalAccessibilityManager.current?.isEnabled != true && animatorScale > 0f) and substitute snap() or tween(0) for all animation specs when animations should be reduced, ensuring the same state changes occur but without motion.

Accessibility Consideration 2: Ensure animated content is never the only communication channel

The favourite toggle must not rely solely on the colour-change animation to communicate its state — the icon itself must change (outlined → filled) and the contentDescription must update ('Add to favourites' → 'Remove from favourites'). A screen reader user who has animations turned off must still understand the state change through the icon variant and its description. Similarly, the expand/collapse state must have a clear semantic indication (e.g., Modifier.semantics { stateDescription = if (isExpanded) 'Expanded' else 'Collapsed' }) so accessibility services can announce the change without relying on the visual animation.

5. Wireframe Sequence & Storyboard

5.1 Scenario: Card Expand Animation — Three States

State	Visual Description	Composables Active	Modifiers / APIs
BEFORE (collapsed)	Card shows hero image (180dp), city name (titleLarge), country + rating row. Description and extra info hidden. Card elevation = 4dp. Background = colorScheme.surface.	Card, Box (image+overlay), Column (title+country+rating), IconButton (favourite)	Modifier.animateContentSize on Column; no expanded content in tree
DURING (animating)	Card height increasing smoothly (spring bounces slightly). Background crossfading surface→surfaceVariant. Content alpha of description rising 0→1. Elevation animating 4dp→8dp. Favourite icon at rest (not triggered).	Same as before + conditional description Column entering tree	animateContentSize active; updateTransition mid-interpolation; Modifier.alpha(contentAlpha) on description
AFTER (expanded)	Full card visible: hero image + city name + country/rating + Divider + full description paragraph + best season Text + 'View Details' OutlinedButton. Background = colorScheme.surfaceVariant. Elevation = 8dp.	All of above + description Column + Divider + seasonText + OutlinedButton	animateContentSize settled; Transition in EXPANDED state; all animated values at target

5.2 Annotated Composable Storyboard Code

```
// BEFORE state (isExpanded = false)
Card(
    modifier = Modifier
        .fillMaxWidth()
        .animateContentSize(animationSpec = spring(
            dampingRatio = Spring.DampingRatioMediumBouncy,
            stiffness = Spring.StiffnessLow
        ))
        .clickable { viewModel.toggleExpanded(id) }
) {
    Column {
        HeroImage(...)          // always visible
        CardHeader(...)         // always visible: title + country + rating
        // No expanded content in tree yet — animateContentSize handles
        // the jump from 0px to content height automatically
    }
}

// DURING/AFTER state (isExpanded = true)
// Transition drives: background color, elevation, content alpha
val transition = updateTransition(targetState = cardState, label = "card")
val bgColor by transition.animateColor(label = "bg") { state ->
```

```
    when (state) {
        CardState.COLLAPSED -> MaterialTheme.colorScheme.surface
        CardState.EXPANDED  -> MaterialTheme.colorScheme.surfaceVariant
    }
}
val contentAlpha by transition.animateFloat(label = "alpha") { state ->
    if (state == CardState.EXPANDED) 1f else 0f
}

// Inside card Column (expanded state adds these):
if (isExpanded) {                          // AnimatedVisibility alternative
    Divider(modifier = Modifier.alpha(contentAlpha))
    Text(
        text     = destination.description,
        modifier = Modifier.alpha(contentAlpha)  // fades in as card opens
    )
    OutlinedButton(onClick = onViewDetails) {
        Text("View Full Details")
    }
}
```

6. Reflection

Designing animations before writing code transforms them from arbitrary eye-candy into intentional communication tools. The process of filling in the animation plan table — specifying trigger, type, affected elements, and timing for each scenario — forces a clear answer to the question 'what is this animation communicating?' before any code is written. For example, deciding that the card expand uses a spring with medium bounce (rather than a linear tween) communicates 'this is an elastic, tactile interaction' rather than 'this is a mechanical transition' — a design decision that shapes the entire feel of the screen. Writing code without this design step typically produces animations that feel arbitrary, inconsistent in timing, or gratuitous. The biggest pitfall of overusing animation is exactly that: motion that exists without communicating anything specific trains users to ignore it, and worse, it can cause cognitive overload when multiple simultaneous animations compete for attention. The guiding test for deciding when not to animate is: 'Does this motion help the user understand a state change, navigate the UI, or feel tactile feedback?' If the answer is no — if the animation is purely decorative or exists because it was technically possible — it should be removed. Practical examples of when not to animate: background colour changes on idle components, text style changes during typing (distracting), and entrance animations on every single list item simultaneously (creates visual chaos).

TASK 3 — Theme + Animation Integration: Building a Polished UI Module

1. Module Definition

Module: Travel Destination Card Gallery — Themed, Animated, Stateful

This module is the primary browsing screen of WanderApp: a scrollable gallery of destination cards that integrates the full WanderApp theme system (colorScheme, typography, shapes) and a coordinated set of animations (card expand/collapse, favourite toggle, filter-driven list entry/exit, and theme switching). It is the most visually prominent screen in the app and serves as the proof-of-concept for how theming and animation combine to create a consistent, premium-feeling experience. Functionally, it allows users to explore destinations, save favourites, read descriptions, and filter by cost tier — all with smooth, responsive visual feedback at every interaction point.

2. Data Model & State Variables

2.1 Data Class

```
data class TravelDestination(
    val destinationId: String,
    val cityName: String,
    val country: String,
    val imageUrl: String,
    val description: String,
    val bestSeason: String,
    val averageCostTier: Int,   // 1=Budget, 2=Mid, 3=Luxury
    val rating: Float,
    val isFavourite: Boolean = false
)
```

2.2 State Variables

```
// ── ViewModel state (survives config change) ──────────────────────
private val _destinations = MutableStateFlow(sampleDestinations)
val destinations = _destinations.asStateFlow()

private val _expandedIds = MutableStateFlow<Set<String>>(emptySet())
val expandedIds = _expandedIds.asStateFlow()

private val _isDarkTheme = MutableStateFlow(false)
val isDarkTheme = _isDarkTheme.asStateFlow()

// ── In-composable animation state (per card) ──────────────────────
val isExpanded: Boolean          // from expandedIds
val isFavourite: Boolean         // from destinations StateFlow
val cardState: CardState         // COLLAPSED or EXPANDED enum

// ── Derived animated values (per card composable) ─────────────────
val transition = updateTransition(cardState, label = "card")
val backgroundColor by transition.animateColor(...)    // surface ↔ surfaceVariant
val cardElevation by transition.animateDp(...)          // 4dp ↔ 8dp
val contentAlpha by transition.animateFloat(...)        // 0f ↔ 1f
```

```
// ── Favourite animation state ──────────────────────────────────────
var favouriteAnimTrigger by remember { mutableStateOf(false) }
val heartScale by animateFloatAsState(
    targetValue = if (favouriteAnimTrigger) 1.3f else 1.0f,
    animationSpec = keyframes { ... }
)
```

2.3 Theme Token → Visual State Mapping

Visual State / Element	Theme Token Used	Notes
Card background (collapsed)	colorScheme.surface	Default card background
Card background (expanded)	colorScheme.surfaceVariant	Animated via updateTransition
Favourite icon (unfavourited)	colorScheme.onSurface with alpha 0.6	Muted outline heart
Favourite icon (favourited)	colorScheme.error (red)	Filled heart, animateColorAsState
Card title text	colorScheme.onSurface + typography.titleLarge	Always high-contrast
Description text	colorScheme.onSurfaceVariant + typography.bodyMedium	Softer tone for secondary info
Cost chip (Budget)	colorScheme.tertiaryContainer with label onTertiaryContainer	Green-amber tonal chip
Cost chip (Luxury)	colorScheme.primaryContainer with label onPrimaryContainer	Blue tonal chip
Filter chip (selected)	colorScheme.secondaryContainer	Teal background when active
View Details button	colorScheme.primary + typography.labelLarge	OutlinedButton with primary border
Screen background	colorScheme.background	Applies to Scaffold background
AppBar	colorScheme.primary + typography.titleLarge	TopAppBar M3 default tokens

3. UI Composition Plan

3.1 Full Composable Hierarchy

```
WanderAppTheme(isDarkTheme = isDarkTheme) {          // MaterialTheme wrapper
    Scaffold(
        topBar = {
```

```
            TopAppBar(
                title = { Text('WanderApp', style = typography.titleLarge) },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = colorScheme.primary,
                    titleContentColor = colorScheme.onPrimary
                ),
                actions = { ThemeToggleSwitch(isDarkTheme, onToggle) }
            )
        },
        bottomBar = { WanderNavigationBar(...) }
    ) { padding ->
        Column(modifier = Modifier.padding(padding)) {

            // ── Filter chip row ──────────────────────────────────
            LazyRow(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                items(filterOptions) { option ->
                    FilterChip(
                        selected = activeFilter == option,
                        onClick  = { activeFilter = option },
                        shape    = MaterialTheme.shapes.small  // 8dp
                    )
                }
            }

            // ── Destination list ─────────────────────────────────
            LazyColumn {
                items(displayedDestinations, key = { it.destinationId }) { dest ->
                    AnimatedVisibility(     // Enter/exit per item
                        visible = true,     // controlled by filter
                        enter = fadeIn(tween(300)) + slideInVertically(tween(250)),
                        exit  = fadeOut(tween(200)) + slideOutHorizontally(tween(200))
                    ) {
                        DestinationCard(
                            destination   = dest,
                            isExpanded    = dest.destinationId in expandedIds,
                            onExpandToggle = {
viewModel.toggleExpanded(dest.destinationId) },
                            onFavToggle    = {
viewModel.toggleFavourite(dest.destinationId) }
                        )
                    }
                }
            }
        }
    }
}
```

4. Implementation Timeline & Dependencies

4.1 Implementation Steps

Step	Task	Deliverable	Est. Time
Step 1	Theme setup	Color.kt + Typography.kt + Shapes.kt + Theme.kt; WanderAppTheme wraps MainActivity	2 hours
Step 2	Static UI layout	DestinationCard without animation; TopAppBar + LazyColumn + FilterChips; all theme tokens applied	3 hours
Step 3	Card expand animation	updateTransition on CardState; animateContentSize; contentAlpha; bg color transition	2 hours
Step 4	Favourite animation	animateFloatAsState with keyframes for scale; animateColorAsState for tint; contentDescription update	1.5 hours
Step 5	List enter/exit animation	AnimatedVisibility per LazyColumn item; filter lambda drives list recomposition	1.5 hours
Step 6	Theme switcher	isDarkTheme StateFlow; ThemeToggleSwitch composable; smooth background transition	1 hour
Step 7	Performance audit	Profile with Android Studio Layout Inspector; verify no jank on expand; check graphicsLayer usage	1.5 hours
Step 8	Accessibility audit	Check contentDescription on all icons; test with TalkBack; verify animate=false path	1 hour

4.2 Dependencies

Library	Version	Purpose
Jetpack Compose BOM	2024.09.00	Core Compose, Material 3, Animation APIs
coil-compose	2.6.0	AsyncImage for destination photos
androidx.lifecycle:lifecycle-viewmodel-compose	2.8.x	ViewModel + collectAsStateWithLifecycle
androidx.navigation:navigation-compose	2.8.x	NavHost for detail screen navigation
kotlinx.coroutines	1.8.x	StateFlow, update {}
androidx.window:window	1.3.x	WindowSizeClass for responsive column count

5. Quality, Accessibility & Performance Checklist

5.1 Pre-release Checklist

Area	Check Item	Pass Criteria
Theme tokens	No raw Color() values in any composable file	Zero occurrences of Color(0xFF...) outside Color.kt
Theme tokens	No hard-coded TextStyle in composables	All text uses MaterialTheme.typography.*
Theme tokens	All shape references use MaterialTheme.shapes.*	No RoundedCornerShape(n) in composables directly
Dark mode	All screens tested in dark theme	No white text on white bg; no black on black; no hard-coded colors breaking
Dynamic colour	Dynamic colour tested on Android 12+ emulator with coloured wallpaper	App palette adapts; no clashing colours with surface tokens
Animation performance	Expand animation runs at 60fps on mid-range device	No frame drops > 16ms in systrace during card expand
Animation performance	LazyColumn scroll is smooth with 10 animated cards	No jank during rapid scroll; off-screen cards not animating
Reduced motion	All animations skip on animatorDurationScale = 0	Content still correct without motion; no missing state changes
Accessibility	All interactive icons have non-empty contentDescription	TalkBack announces all actions correctly
Accessibility	Expanded/collapsed state described semantically	Modifier.semantics { stateDescription } set on cards
Image loading	All Coil requests have placeholder + error drawable	No blank boxes on slow network or broken URL
List performance	key parameter set on all LazyColumn items	No incorrect state attached to wrong item after filter

5.2 Edge Case Test Cases

#	Edge Case	Steps to Reproduce	Expected Behaviour	Actual Result / Notes
EC-1	Theme change while card expand animation in progress	1. Tap a card to start expand animation. 2. Immediately toggle dark/light theme switch before animation completes.	Animation continues to completion with new theme colours from mid-point; no colour jump or visual glitch; final state uses new theme tokens correctly.	updateTransition handles mid-animation interruption gracefully. The colorScheme change triggers immediate colour recalculation in

#	Edge Case	Steps to Reproduce	Expected Behaviour	Actual Result / Notes
				the animateColor interpolation.
EC-2	Rapid favourite toggle on slow device	1. On a low-end emulator (CPU throttled). 2. Tap favourite icon 10 times quickly.	isFavourite reflects final correct state (even/odd taps); no duplicate animation triggers; icon settles correctly to final state.	StateFlow.update {} is thread-safe and atomic; final value is always correct. Animation: if new toggle fires before previous completes, scale jumps to new targetValue from current position — acceptable.
EC-3	All cards expanded simultaneously	1. Expand all 10 cards one by one. 2. Observe memory and rendering.	All cards remain expanded with correct colours; no layout corruption; scroll remains smooth.	Each card's transition is independent. 10 simultaneous expanded states is within normal Compose rendering budget.
EC-4	Filter applied while card is mid-expand	1. Start expanding a card. 2. Immediately tap a filter chip that removes that card.	AnimatedVisibility exit animation plays for the card; card slides out smoothly; no crash or orphaned animation.	AnimatedVisibility handles this correctly — when visible=false, the exit animation runs and content is disposed afterward.

6. Reflection

The biggest challenge in integrating theming and animation was ensuring that animated colour values always reflected the current theme rather than a stale snapshot. When updateTransition interpolates a card's background colour from surface to surfaceVariant, both of those colour values must be read from MaterialTheme.colorScheme at the time of the animation frame — if they were captured as a val outside the transition lambda, they would not update when the user switches dark/light mode mid-animation. The solution (reading colorScheme inside the animate* lambdas where they are always evaluated within the current composition context) was not obvious without encountering the bug first. Together, theming and animation improve user satisfaction in ways that neither can achieve alone: consistent theming creates brand trust and visual coherence (the app feels like one product, not a collection of screens), while animation creates the sense of physicality and responsiveness that makes interactions feel rewarding rather than mechanical. A user who taps a card and sees nothing happen for a frame might wonder if the tap registered; the same tap with an immediate spring animation communicates instantaneous response. A future developer working on this codebase should be cautioned against three things: animating properties that are not theme-aware (hard-coded hex values inside animation specs will not follow theme changes), adding animations to too many simultaneous elements (the Destinations screen already has card expand, favourite, and list-enter animations — adding more would create visual chaos), and ignoring the animatorDurationScale system setting (an

app that does not respect the accessibility setting is inaccessible to users who need reduced motion for vestibular disorders).

TASK 4 — Capstone: Build, Document & Present Themed Animated Feature

1. Implementation Snapshot

1.1 What Was Built

The Travel Destination Card Gallery module was implemented with the following components working end-to-end: the complete WanderAppTheme with light and dark ColorScheme, WanderTypography, and WanderShapes; the TopAppBar and NavigationBar using theme tokens; the DestinationCard composable with updateTransition-driven expand/collapse animation; the favourite toggle with keyframes scale animation; filter chips with AnimatedVisibility-driven list entry/exit; and the theme toggle switch. All composables reference theme tokens exclusively — zero hard-coded colour or text style values appear outside the theme files.

1.2 Screenshot Descriptions

Screenshot	Theme	State	Key Visual Elements Visible
SS-1: List Screen Light	Light (#F8FAFB bg)	All cards collapsed	AppBar in #1565C0 blue; white cards with 12dp corners; Montserrat city names; teal filter chips; outline heart icons
SS-2: List Screen Dark	Dark (#0D0D1A bg)	All cards collapsed	AppBar in dark primary #90CAF9 text; dark surface cards #1A1A2E; same layout, inverted palette; filled heart in red on favourited card
SS-3: Card Expanded (Light)	Light	Kyoto card expanded	Card bg shifted to #DDE3EA surfaceVariant; description text visible; rating stars row; 'View Full Details' outlined button; elevation shadow visible
SS-4: Favourite Animate (Dark)	Dark	During heart tap animation	Heart icon at peak scale 1.3x; color mid-transition white→red; surrounding cards unchanged

2. Implementation Documentation

2.1 Theme Tokens Used

Component	Token Used	Value (Light / Dark)
TopAppBar background	colorScheme.primary	#1565C0 / #90CAF9
Card background (collapsed)	colorScheme.surface	#FFFFFF / #1A1A2E

Component	Token Used	Value (Light / Dark)
Card background (expanded)	colorScheme.surfaceVariant	#DDE3EA / #3A3A4A
City name text	typography.titleLarge + colorScheme.onSurface	22sp Montserrat Medium
Description text	typography.bodyMedium + colorScheme.onSurfaceVariant	14sp Roboto Regular
Card shape	shapes.medium	RoundedCornerShape(12dp)
Favourite icon (active)	colorScheme.error	#B3261E / #F2B8B5
Filter chip (selected)	colorScheme.secondaryContainer	#B2DFDB / #003731
Screen background	colorScheme.background	#F8FAFB / #0D0D1A

2.2 Key Code Snippets

```
// Theme.kt — final implementation
@Composable
fun WanderAppTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            if (darkTheme) dynamicDarkColorScheme(LocalContext.current)
            else dynamicLightColorScheme(LocalContext.current)
        }
        darkTheme -> WanderDarkColorScheme
        else      -> WanderLightColorScheme
    }
    MaterialTheme(colorScheme = colorScheme, typography = WanderTypography,
                  shapes = WanderShapes, content = content)
}

// DestinationCard.kt — updateTransition for coordinated animation
@Composable
fun DestinationCard(destination: TravelDestination, isExpanded: Boolean, ...) {
    val cardState = if (isExpanded) CardState.EXPANDED else CardState.COLLAPSED
    val transition = updateTransition(cardState, label =
"card_${destination.destinationId}")

    val bgColor by transition.animateColor(
        transitionSpec = { spring(stiffness = Spring.StiffnessMedium) },
        label = "bg_${destination.destinationId}"
    ) { when (it) {
        CardState.EXPANDED  -> MaterialTheme.colorScheme.surfaceVariant
        CardState.COLLAPSED -> MaterialTheme.colorScheme.surface
    }}
```

```
    val elevation by transition.animateDp(
        transitionSpec = { tween(200) },
        label = "elev"
    ) { if (it == CardState.EXPANDED) 8.dp else 4.dp }

    Card(
        shape    = MaterialTheme.shapes.medium,
        colors   = CardDefaults.cardColors(containerColor = bgColor),
        elevation = CardDefaults.cardElevation(elevation),
        modifier = Modifier.animateContentSize(
            animationSpec = spring(dampingRatio = Spring.DampingRatioMediumBouncy)
        )
    ) { /* card content */ }
}
```

2.3 Deviations from Blueprint & Reasons

Blueprint Plan	What Was Implemented	Reason for Deviation
AnimatedVisibility per list item for filter enter/exit	AnimatedVisibility kept but only for enter; exit relies on LazyColumn item removal with key	LazyColumn item removal with key provides adequate exit; full AnimatedVisibility exit caused layout inconsistency with remaining items' positions
Rating stars reveal with slideInHorizontally + delay	Rating stars use simple fadeIn only; no slide	Slide caused visual clipping outside card bounds during expand; fade achieves same entrance effect without clipping issue
Error field shake animation in filter screen	Deferred to v2 — not implemented	Out of scope for this module's card gallery; will be added to form screens in a future session

2.4 Performance Evaluation

- Card expand animation: No jank observed on Pixel 6 (API 33). Android Studio Profiler shows expand frames consistently under 16ms. The spring animation on animateContentSize generates ~18 frames at 60fps, all within budget.
- Favourite icon animation: keyframes animation runs entirely on the graphics layer via graphicsLayer(scaleX, scaleY) — zero layout passes triggered during the 300ms animation. No jank.
- Theme switching: When isDarkTheme flips, all animated card backgrounds transition smoothly using tween(400) on their animateColor. The transition is smooth with no flicker. On older devices (API 26), the transition is slightly faster due to fewer animating composables.
- Potential improvement: The LazyColumn currently recomposes all visible cards when any single card's state changes (because destinations is a List, and any update creates a new list reference). Converting to use items(key = { it.destinationId }) with stable keys already mitigates most of this, but moving to LazyPagingItems would eliminate the remaining unnecessary recompositions on large lists.

3. Presentation Slide Deck

Slide 1 — Feature Overview & Why It Matters

Title: WanderApp — Travel Destination Card Gallery

What: A themed, animated, stateful scrollable card gallery for browsing travel destinations. Who: Travellers who want to save favourites and read destination details. Why it matters: This module is the first screen users interact with — it sets the quality bar for the entire app. A polished, smooth, brand-consistent first screen builds user trust and communicates that the app is professionally crafted. Key numbers: 7 coordinated animations, 1 complete theme system (light/dark/dynamic), 0 hard-coded colour values.

Slide 2 — Theming + UI Preview

Theme system: WanderAppTheme wraps entire app → WanderLightColorScheme (12 tokens) + WanderDarkColorScheme (12 tokens) + WanderTypography (8 styles: Montserrat for headers, Roboto for body) + WanderShapes (6 tokens: 4dp to 28dp). Brand identity: Deep ocean blue primary, teal-green secondary, amber tertiary. Card design: 12dp medium corners (shapes.medium), surface/surfaceVariant colour shift on expand, Montserrat titleLarge for city names. Dynamic colour: On Android 12+, the palette adapts to the user's wallpaper colours automatically.

Slide 3 — Animation Flow + Lessons Learned

Animation inventory: (1) Card expand — updateTransition drives background colour, elevation, content alpha in lockstep with spring(MediumBouncy). (2) Favourite toggle — keyframes scale 1.0→1.3→1.0 in 300ms via graphicsLayer; animateColorAsState for tint. (3) List entry — AnimatedVisibility with fadeIn+slideInVertically per LazyColumn item. Lessons learned: (1) Read theme colours inside transition lambdas — never outside them — or dark/light switching breaks mid-animation colours. (2) graphicsLayer for non-layout animations is essential for smooth performance. (3) Accessibility first: all animations must have a static fallback path.

Slide 4 (Optional) — Next Steps & Future Enhancements

v2 Enhancements planned: (1) Animated favourite count badge on NavigationBar tab icon — badge slides and scales in when count increments. (2) Dynamic colour support for older devices via palette extraction from loaded destination images using the Palette API — generates a personalised warm/cool tone for each destination card. (3) Shared element transitions between the card list and the detail screen — the hero image morphs from its card position to full-screen in the detail view. (4) Shimmer loading animation using SubcomposeAsyncImage with a shimmer Modifier while images load from Coil.

4. Reflection & Improvement Log

The most surprising discovery during implementation was how much the order of operations matters when combining updateTransition with MaterialTheme. Initially, the card background colour was calculated outside the transition lambda using val expandedColor = MaterialTheme.colorScheme.surfaceVariant above the Card composable. This worked in light mode but produced a stale colour snapshot in dark mode after theme switching — the transition interpolated between the old light surfaceVariant and the new dark surfaceVariant, creating an unintended colour flash. Moving the colorScheme reads inside the animate* lambdas (where they are evaluated in every recomposition frame) resolved the issue. This taught a concrete rule: in Compose, dynamic values that depend on composition context (like theme colours) must be read inside lambdas that are evaluated during composition, not captured as val before the lambda. For future features with theme + animation, the workflow established in this session will be followed: (1) define all theme tokens first, (2) build static

UI referencing only tokens, (3) add animations one at a time (not all at once), and (4) test the Reduce Motion path before the standard path, to ensure the feature works correctly without animation before optimising the animated version. What would be done differently: the card expand animation's content alpha was added after the basic expand was working, creating a temporary visual inconsistency during development. In future, all coordinated animations (size, colour, alpha, elevation) would be planned and implemented together in the updateTransition from the start.

Future Enhancements:

- Enhancement 1: Animate the favourite count badge on the NavigationBar bottom tab — when the count increments, the badge number slides out upward and the new number slides in from below (using AnimatedContent with verticalScroll exit/enter), combined with a brief scale pulse (animateFloatAsState, 1.0→1.2→1.0 in 200ms). This would make the favourite-adding action feel globally acknowledged, not just locally on the card.
- Enhancement 2: Dynamic colour extraction from loaded destination images using the Palette API (androidx.palette:palette) — when Coil finishes loading a destination's hero image, the dominant Swatch colour is extracted and fed into dynamicColorScheme() to temporarily tint the expanded card's surfaceVariant with a personalised accent derived from the destination's actual photo. This creates a unique, photo-responsive card aesthetic for each destination.

Session 12 — Learning Summary

Consolidated key learnings from all four tasks in Session 12 — Advanced Theming, Animation Design, and Polished UI Module Integration in Jetpack Compose.

Key Concepts Mastered

Concept	What I Learned	Where Applied
colorScheme (M3)	Named colour roles; propagates to all components from MaterialTheme	Task 1, 3, 4 — WanderColorSchemes
Dynamic colour	dynamicLightColorScheme() on Android 12+; wallpaper-personalised palette	Task 1, 4 — WanderAppTheme
Typography tokens	Named text styles; Montserrat + Roboto font families in WanderTypography	Task 1, 3 — all text composables
Shapes tokens	extraSmall to extraLarge (4dp–28dp); WanderShapes object	Task 1, 3 — cards, buttons, chips
Theme file architecture	Color.kt + Typography.kt + Shapes.kt + Theme.kt separation	Task 1, 3, 4 — full theme structure
animateFloatAsState	Smoothly interpolates Float; keyframes for bounce; graphicsLayer for GPU perf	Task 2, 3 — favourite heart scale
animateColorAsState	Interpolates Color values; reads theme tokens inside lambda for correctness	Task 2, 3 — card bg, icon tint
AnimatedVisibility	enter/exit transition specs; fadeIn+slideIn; handles LazyColumn item lifecycle	Task 2, 3 — list item enter/exit
animateContentSize	Modifier for smooth height changes; spring spec for bounce	Task 2, 3, 4 — card expand
updateTransition	Coordinates multiple animations on same state; transition.animateColor/Dp/Float	Task 2, 3, 4 — card expand system
Theme tokens in animation lambdas	Must read colorScheme inside animate* lambdas, not outside; avoids stale snapshot	Task 4 — key bug discovered
Reduce Motion accessibility	animatorDurationScale = 0; substitute snap() for all specs; content must work sans motion	Task 2, 3 — accessibility checklist

Best Practices Identified

- Never use raw Color() or hardcoded TextStyle values in composables — always reference MaterialTheme.colorScheme.* and MaterialTheme.typography.*.
- Read theme colour values inside animate* lambdas (not captured as val outside) to ensure dark/light mode switches correctly update mid-animation.
- Use updateTransition when multiple animated values change in response to the same state — it guarantees they progress in lockstep.
- Apply animations via Modifier.graphicsLayer for scale/rotation/alpha — never via Modifier.size for non-layout animations; graphicsLayer avoids layout passes.
- Always implement the Reduce Motion path (snap() or tween(0)) before adding motion — motion should enhance a functional feature, not be required for correctness.
- Every interactive animated element must have its own semantic state description — contentDescription alone is not sufficient for expand/collapse state.
- Define animation specs as top-level val constants to avoid creating new Spring/Tween objects on every recomposition.
- Follow theme → static layout → animation order: never add animations before the static layout is correct and all theme tokens are in place.

Student Declaration

I, Sudarshan G, confirm that all work submitted in this assessment document is my own. The theme designs, code examples, animation plans, module blueprints, and reflections represent my own understanding of the concepts covered in Session 12 of the MindMatrix Android Development programme.

Student: Sudarshan G

Session: Session 12 — Theming, Animation & Polished UI in Compose

Programme: Android App Development with Kotlin & Jetpack Compose